

115 System Theory — Lab Session Ch.4 Homework

Building Classic Agent Paradigms

111061540 張晉承

1. Compare the Three Paradigms

Paradigm	Main idea	Strength	Typical use
ReAct	Interleave reasoning and acting in a $Thought \rightarrow Action \rightarrow Observation$ loop: $(th_t, a_t) = \pi(q, (a_1, o_1), \dots, (a_{t-1}, o_{t-1}))$, $o_t = T(a_t)$. The agent decides the next step only after seeing the result of the previous one.	Dynamic adaptation and error correction from real-time feedback; tool synergy (search for fresh knowledge, calculator for exact arithmetic); highly interpretable trace.	Tasks full of uncertainty that need external tools/APIs: real-time information lookup (weather, news, stocks), database/API operations, delegating precise calculations.
Plan-and-Solve	Think before acting. First generate a complete plan $P = \pi_{plan}(q) = (p_1, \dots, p_n)$, then execute it step by step: $s_i = \pi_{solve}(q, P, (s_1, \dots, s_{i-1}))$.	Global, structured view of the task; keeps multi-step reasoning on track (avoids CoT "going off track"); stable, predictable execution with fewer chances of drifting into local optima.	Tasks with a clear logical path: multi-step math word problems, report writing that integrates several sources, code generation (design functions/classes/modules first), scheduling.
Reflection	$Execute \rightarrow Reflect \rightarrow Refine$. Produce a first draft O_i , have a reviewer generate feedback $F_i = \pi_{reflect}(\text{task}, O_i)$, then refine $O_{i+1} = \pi_{refine}(\text{task}, O_i, F_i)$, iterating until good enough.	Internal self-correction loop that can fix higher-level logic and strategy errors; continuously raises answer quality and success rate; keeps a short-term memory of how the draft improved.	Tasks with very high requirements on quality/reliability: code generation and debugging, academic/technical writing, answers that must be checked for factual or logical errors.

2. Paradigm Matching

Scenario	Paradigm
A. Smart-home assistant: "Is the bedroom window open? If it is open, close it."	ReAct
B. "Plan a two-day study schedule for my upcoming exams."	Plan-and-Solve
C. LLM generates a Python program, runs it, observes an error, and revises the program.	Reflection

A → ReAct

The assistant cannot know the window state in advance; it must interact with the environment and decide what to do based on what it observes. The next action depends on the observation:

```
Thought: I need to know the current state of the bedroom window.  
Action: CheckSensor[bedroom_window]  
Observation: open  
Thought: The window is open, so I should close it.  
Action: CloseWindow[bedroom_window]  
Observation: success  
Thought: The window is now closed.  
Action: Finish[The bedroom window was open and has now been closed.]
```

The task is short, depends on external tools (sensor reading, actuator control), and contains a conditional branch ("if it is open..."), so a fixed plan made beforehand is not suitable. This is exactly the dynamic Thought–Action–Observation loop of ReAct. (If the window were already closed, the agent would simply finish after the first observation.)

B → Plan-and-Solve

Building a study schedule is a task with a clear structure and no need for real-time external feedback. It benefits from a global view first, then filling in details step by step:

1. **Plan:** list the subjects and exam dates → estimate the workload and priority of each subject → split the two days into time blocks (study, review, breaks) → allocate subjects to blocks.
2. **Solve:** carry out each step in order, using the results of earlier steps (e.g. priorities decide how many blocks each subject gets), and output the final timetable.

Planning first keeps the schedule balanced and consistent overall (no subject forgotten, no overlapping time slots), which a step-by-step reactive approach might not guarantee.

C → Reflection

This is the *Execute* → *Reflect* → *Refine* loop:

- **Execute:** the LLM generates a first version of the program O_1 .
- **Reflect:** the program is run and the error message/traceback is analyzed as feedback F_1 (what went wrong and why).
- **Refine:** the LLM uses the original task, the previous code O_1 , and the feedback F_1 to produce a revised program O_2 ; the loop repeats until the program runs correctly.

The aim is to iteratively improve the quality and correctness of one artifact (the code) through criticism and correction, which is the core of Reflection. Running the code is a tool call, as in ReAct, but here the observation is used to *critique and rewrite the whole output* rather than to choose the next exploratory action, so Reflection is the best match.